

Práctica 4.1: Manejo de sockets y explotación de concurrencia en Python

Índice

1	Objetivos	1
2	Preparación del entorno de trabajo	1
3	Ejercicios	2
	Ejercicio 1. Concurrencia en Python usando <code>threading</code> y <code>multiprocessing</code>	2
	Limitaciones del intérprete de Python y casos de uso	2
	Modo de uso	3
	Efectos del GIL en el tiempo de ejecución	4
	Ejercicio 2. Sincronización entre hilos en Python	5
	Ejercicio 3. Uso práctico de sockets, procesos e hilos para comunicación y procesamiento en entornos cliente-servidor	6
	Ejercicio 4. Despliegue de cliente-servidor en nodos diferentes	7

1 Objetivos

El principal objetivo de esta práctica es familiarizarse con el manejo básico de hilos, procesos y sockets en Python. Como parte de la práctica se proporciona un breve tutorial que ilustra las diferencias entre los hilos y los procesos en Python, en cuanto a programación, limitaciones y casos de uso típicos.

En el fichero comprimido con el código y otros recursos de la práctica se proporciona una carpeta con el código fuente asociado a cada ejercicio, así como un proyecto de VSCode y los ficheros necesarios para construir una imagen de Docker que se utilizará para realizar la práctica con el proyecto VSCode proporcionado.

2 Preparación del entorno de trabajo

Para poder trabajar en la práctica es preciso tener un equipo con Docker instalado. La máquina virtual VMware de Debian disponible en el campus virtual puede usarse para ello, ya que incluye Docker preinstalado. No obstante, los estudiantes que lo deseen pueden simplemente utilizar su portátil con Docker instalado (sin la citada máquina virtual), siendo Docker Desktop lo que se ha de instalar sobre Mac OS o Windows nativo.

Como se indicó anteriormente, entre los ficheros de la práctica se proporcionan dos componentes básicos que se usarán sobre Docker:

- *DockerContainer*: Este directorio contiene los ficheros necesarios para construir la imagen `ray-os` que se usará para las últimas prácticas. Esta imagen contendrá un entorno funcional de Python con todas las dependencias necesarias instaladas.
- *PythonProject*: Es un proyecto VSCode basado en DevContainers. Para usarlo es preciso construir antes la imagen de Docker `ray-os`.

El primer paso para preparar el entorno de trabajo consiste en crear la imagen de Docker. Para ello abriremos una terminal, cambiaremos al directorio *DockerContainer* y, desde ese directorio, ejecutaremos el siguiente comando:

```
$ docker image build -t ray-os .
```

La creación de esta imagen puede llevar bastante tiempo (al menos 20 minutos) ya que requiere descargar la imagen de una versión reciente de Debian, sobre la que se instalan una serie de utilidades esenciales, así como un *virtual environment* de Python con las dependencias necesarias. Como aspecto positivo, una vez creada la imagen, será posible crear múltiples contenedores con ella, como el que requiere el proyecto de VSCode.

Tras crear la imagen podremos trabajar en la práctica con el proyecto *PythonProject* donde debemos mover o copiar las carpetas de los distintos ejercicios de la práctica. Finalmente, abriremos el proyecto con VSCode. Al hacer esto pedirá arrancar un contenedor tras la apertura del mismo. Como en el proyecto DevContainers facilitado para trabajar en las prácticas del Módulo 1, la raíz del proyecto *PythonProject* funciona como una carpeta compartida con nuestro *host*. Todo fichero que alteremos/añadamos allí estará disponible desde el sistema operativo anfitrión.

Muy importante:

Una vez arrancado el contenedor con VSCode, debemos hacer dos cosas más para que el proyecto esté totalmente operativo:

1. Abrir una ventana de terminal y crear un enlace simbólico para que VSCode trabaje por defecto con el *virtual environment* de Python ya instalado en el contenedor. Para ello ejecutaremos el siguiente comando:

```
$ ln -s /root/venv .venv
```

2. Por último esperaremos a que VSCode acabe de instalar una serie de extensiones de VSCode, cuya instalación se inicia automáticamente al abrir el proyecto por primera vez. Sabremos que la instalación de estas extensiones ha terminado cuando desaparezca el icono del reloj de la barra lateral izquierda del entorno donde se accede a las extensiones de VSCode.

3 Ejercicios

Ejercicio 1. Concurrencia en Python usando **threading** y **multiprocessing**

Para explotar concurrencia en los programas, Python ofrece dos módulos básicos: **threading** y **multiprocessing**. Aunque ambos permiten la ejecución simultánea de tareas, se basan en modelos diferentes. El módulo **threading** permite ejecutar varios hilos (*threads*) dentro de un mismo proceso –intérprete de Python–, compartiendo memoria. Por contra, **multiprocessing** permite la creación de varios procesos (intérpretes) independientes, cada uno con su propia imagen de memoria.

Mediante estos módulos podemos explotar concurrencia para acceder a cierta funcionalidad de gran utilidad como:

- Aprovechar múltiples núcleos de la CPU
- Realizar tareas que se bloquean esperando por entrada-salida sin detener todo el programa
- Modelar sistemas con componentes independientes que se comunican entre sí (como productor-consumidor, cliente-servidor, etc.).

Limitaciones del intérprete de Python y casos de uso

A la hora de explotar concurrencia, además de tener en cuenta las diferencias entre procesos e hilos introducidas en el primer módulo de la asignatura, debemos considerar un aspecto clave de la implementación más extendida de Python, llamada CPython. Esta implementación hace uso de un cerrojo global del intérprete denominado *Global Interpreter Lock* (GIL), que **restringe la ejecución simultánea de múltiples hilos en un único proceso de Python**. Es decir, aunque creemos varios hilos en nuestro programa, **solo uno puede ejecutar bytecode de Python a la vez**. Esto significa que

incluso en máquinas con múltiples núcleos, los hilos de Python **no pueden ejecutar código Python en paralelo** si están haciendo trabajo intensivo de CPU.

El GIL existe por varias razones históricas y técnicas:

- CPython **no tiene un recolector de basura con contadores de referencias seguros en entornos multihilo**. El GIL simplifica la gestión de memoria y evita que haya que usar bloqueos (*locks*) finos alrededor de cada objeto.
- El GIL hace que ciertas operaciones en programas de un solo hilo sean más rápidas y fáciles de mantener.
- Es más sencillo escribir extensiones en C para Python cuando se puede asumir que no hay acceso concurrente.

Debida a la existencia del GIL, los módulos `threading` y `multiprocessing` tienen distintos casos de uso, como recoge la siguiente tabla:

Caso de uso	Módulo Recomendado
El procesamiento de tareas incluye muchas esperas por entrada-salida (p.ej., red, disco, etc.) o sincronización	<code>threading</code>
Las tareas realizan cálculos intensivos en CPU	<code>multiprocessing</code>
Las tareas no son muy intensivas en CPU (tienen esperas) y requieren compartir memoria a la que se accede muy frecuentemente	<code>threading</code>
Se precisa tener aislamiento entre tareas (intérprete de Python propio para cada una)	<code>multiprocessing</code>

Modo de uso

Tanto `threading` como `multiprocessing` ofrecen al programador una API muy similar para la creación, terminación y espera de hilos y procesos, respectivamente. Sendos módulos exportan una clase específica para ello: `Thread` y `Process`. Para crear un hilo o proceso basta crear un objeto de esa clase e invocar el método `start()`. Análogamente el método `join()` permite que otro proceso/hilo se quede esperando a la terminación del proceso/hilo lanzado con `start()`.

Para que el programador indique qué código (función) ejecutará el hilo/proceso a crear existen dos alternativas:

1. Crear una clase extendiendo la clase base correspondiente (`Thread` o `Process`) y definir su método `run()`, donde indicaremos el procesamiento realizado por el hilo.
2. Crear un objeto `Thread` o `Process`, pasando como parámetro en el constructor la función que deseamos que el hilo ejecute, así como sus argumentos.

A continuación se muestra un ejemplo de la primera aproximación, incluyendo la variante con procesos e hilos. En ambos casos, se define una clase `MiTarea` para especificar el procesamiento realizado por el nuevo proceso/hilo.

Procesos

```
from multiprocessing import Process

class MiTarea(Process):
    def __init__(self):
        super().__init__()

    def run(self):
        print("Ejecutando tarea")

if __name__ == '__main__':
    process = MiTarea()
    process.start()
    process.join()
```

Hilos

```

from threading import Thread

class MiTarea(Thread):
    def __init__(self):
        super().__init__()

    def run(self):
        print("Ejecutando tarea")

if __name__ == '__main__':
    thread = MiTarea()
    thread.start()
    thread.join()

```

Para ilustrar el uso de la segunda alternativa, donde se pasa la función del hilo/proceso como argumento, se proporciona un ejemplo adicional a continuación. En este caso el programa crea un hilo y un proceso que ejecutan la misma función:

```

#!/usr/bin/env python
import threading
import multiprocessing
import time

def worker(name):
    print(f"Inicio de {name}")
    time.sleep(2)
    print(f"Fin de {name}")

if __name__ == '__main__':
    # Hilo
    t = threading.Thread(target=worker, args=("Hilo",))
    # Proceso
    p = multiprocessing.Process(target=worker, args=("Proceso",))
    t.start()
    p.start()
    t.join()
    p.join()

```

Efectos del GIL en el tiempo de ejecución

Como se indicó anteriormente el GIL impide la ejecución simultánea de código Python en múltiples hilos del mismo proceso. Para observar este aspecto, se propone la realización del siguiente ejercicio:

1. Ejecuta el siguiente programa (*timing.py*) que temporiza la ejecución de una misma tarea invocada por múltiples hilos y procesos simultáneos:

```

#!/usr/bin/env python
import time
from threading import Thread
from multiprocessing import Process

N = 5

def tarea():
    time.sleep(1)

def program():
    # Con hilos
    inicio = time.time()
    hilos = [Thread(target=tarea) for _ in range(N)]

```

```
[t.start() for t in hilos]
[t.join() for t in hilos]
print("Tiempo con hilos:", time.time() - inicio)

# Con procesos
inicio = time.time()
procesos = [Process(target=tarea) for _ in range(N)]
[p.start() for p in procesos]
[p.join() for p in procesos]
print("Tiempo con procesos:", time.time() - inicio)
```

2. Toma nota de los tiempos mostrados por el programa. ¿Qué alternativa es más rápida?
3. Modifica la función tarea empleando la nueva definición proporcionada a continuación, donde no se hace una espera como antes, sino un cómputo:

```
def tarea():
    suma = 0
    for i in range(10**7):
        suma += i
```

4. Toma nota de los tiempos mostrados por el programa. ¿Qué alternativa es más rápida? ¿Coincide con la respuesta del punto 2? ¿A qué se debe este resultado?

Ejercicio 2. Sincronización entre hilos en Python

A pesar del uso del GIL en la implementación CPython, el acceso atómico a variables y objetos en programas multihilo no está asegurado automáticamente. Para permitir la ejecución segura de métodos y evitar la corrupción de estructuras de datos y variables compartidas durante el acceso concurrente es preciso utilizar mecanismos de sincronización de Python.

El módulo `threading` ofrece implementaciones de semáforos, cerrojos y variables condicionales, representadas mediante las clases `Semaphore`, `Lock` y `Condition`, respectivamente. Una característica distintiva de la API de estos recursos de sincronización es que todos ellos ofrecen las operaciones `acquire()` y `release()`, que tienen diferente propósito en cada uno de los recursos de sincronización, como se resume en la tabla.

Recurso	Operaciones realizadas por <code>acquire()</code> y <code>release()</code> , respectivamente
<code>threading.Semaphore</code>	<code>wait</code> y <code>signal</code> del semáforo
<code>threading.Lock</code>	<code>lock</code> y <code>unlock</code> del cerrojo
<code>threading.Condition</code>	<code>lock</code> y <code>unlock</code> sobre el mutex asociado a la variable condicional.

Para familiarizarse con el uso de primitivas de sincronización en Python se pide que el estudiante realice lo siguiente:

- Con el código de la práctica se proporcionan 3 ejemplos de uso de cerrojos, semáforos y variables condicionales: *lock.py*, *sem.py* y *prodcons.py*. Ejecuta estos ejemplos y analiza la implementación.
 - **Nota:** El constructor por defecto de las variables condicionales (sin argumentos) crea un cerrojo privado para la variable condicional. No obstante, durante la construcción del objeto es posible pasar un cerrojo creado previamente, como en el ejemplo *prodcons.py*, donde esto es necesario al usar múltiples variables condicionales para la implementación.
- Implementa una versión en Python del ejercicio 2A de la práctica 1.2: Inserción alternada de enteros por parte de dos hilos en un vector usando semáforos. En esta implementación se usará una lista de Python, en la que se insertará al final usando el método `append()`.

Ejercicio 3. Uso práctico de sockets, procesos e hilos para comunicación y procesamiento en entornos cliente-servidor

En este ejercicio se pide desarrollar un servidor en Python que implemente un servicio que proporciona la temperatura media mensual en una ciudad. Esta información se ha de inferir de un fichero *temperatures.csv* ya proporcionado, y cuyo contenido se cargará en la memoria del servidor como un `DataFrame` de Pandas. La columna “temp” fichero contiene desglosada la temperatura media de cada día del año 2024 para 4 ciudades: París, Madrid, Berlín, Roma. Los nombres de las ciudades figuran en inglés en la columna “name” del fichero.

El servidor aceptará conexiones y peticiones de los clientes a través de un socket TCP en el puerto 10000. Para hacer una petición, el cliente deberá iniciar una conexión con el servidor, y a continuación enviarle una cadena de caracteres con el formato "<Ciudad>:<NúmMes>". De este modo, la cadena "Madrid:3" indica la petición de la temperatura media en Madrid durante el mes de marzo. Al recibir esta petición, el servidor filtrará las entradas correspondientes del `DataFrame` (fechas del 1-03-2024 al 31-03-2024) y calculará la media de esas entradas, que devolverá al cliente.

En este ejercicio se ha de realizar lo siguiente:

- **(Parte A)** Crear una versión básica del servidor (*basic-server.py*) que solo atienda a un cliente de forma simultánea. Es decir, tras aceptar una nueva conexión, el servidor interactuará con el cliente hasta que cierre la conexión. A continuación el servidor volverá a quedarse en espera de nuevas conexiones.

Para depurar este servidor se puede usar el programa `telnet` como cliente interactivo, a lanzar tras ejecutar el servidor en otra ventana de terminal de la misma máquina o contenedor Docker. Este programa acepta como parámetros el nombre de host o IP del servidor, y el número de puerto:

```
$ telnet localhost 10000
Trying ::1...
telnet: connect to address ::1: Connection refused
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Madrid:12
6.745161290322581°C
Rome:8
28.203225806451616°C
Berlin:4
11.653333333333334°C
Paris:13
Wrong month
Sevilla:2
Unknown city
END
Connection closed by foreign host.
```

Como se puede observar al teclear una cadena correcta, el servidor ha de respondernos con la temperatura media de ese mes. Para las cadenas incorrectas el servidor devuelve un mensaje de error. Al hacer una petición con la cadena “END” se termina la ejecución.

Posible salida en el servidor:

```
$ python3 basic-server.py
Server listening on 0.0.0.0:10000
Accepted connection from ('127.0.0.1', 53945)
Handling connection from ('127.0.0.1', 53945)
Received: Madrid:12
Received: Rome:8
Received: Berlin:4
Received: Paris:13
Received: Sevilla:2
```

- **(Parte B)** Empleando como base el servidor desarrollado en el punto anterior, crear otra versión del servidor (*multithreaded-server.py*) donde un hilo separado se encargará de interactuar con cada cliente. El hilo se creará tras establecer una nueva conexión (cuando la llamada `accept()` retorne) y terminará tan pronto como el cliente cierre la conexión.

Para verificar que el servidor está funcionando correctamente, se pueden invocar varias instancias simultáneas del programa `telnet` con los mismos argumentos pero desde distintos terminales. Esto permitirá emitir peticiones desde distintos clientes, con puertos diferentes asignados automáticamente por el sistema operativo.

- **(Parte C)** Desarrollar otra variante del servidor (*multiprocess-server.py*) donde se creen procesos auxiliares por cliente –en lugar de hilos– para el procesamiento de las peticiones.
- **(Parte D)** Desarrollar un programa Python –llamado *request-generator.py*– que permita hacer peticiones continuas automáticamente al servidor utilizando múltiples hilos concurrentes durante un tiempo T especificado por el usuario. En particular, el programa creará N hilos que actuarán como clientes, realizando cada uno de ellos peticiones secuenciales y aleatorias, introduciendo un retardo de M milisegundos entre ellas. Al finalizar la ejecución, se reportará la tasa de peticiones por segundo observadas durante la misma.

Al lanzar el programa desde línea de comandos se pasarán los siguientes argumentos

```
request-generator.py <host_servidor:puerto> <T> <N> <M>
```

Para el cálculo de la tasa de peticiones por segundo al final de la ejecución, se compartirá una variable entera entre los hilos para llevar la cuenta del número de peticiones realizadas. Este contador se actualizará de forma segura cada vez que algún cliente reciba la respuesta del servidor (valor de temperatura media).

Utilizar el generador de peticiones para determinar qué implementación del servidor (Parte B o C) resulta más eficiente.

Nota: Para esta parte del ejercicio se proporciona un esqueleto del programa en *request-generator.py*

Ejemplo de ejecución (usando otra terminal distinta a la empleada para lanzar el servidor)

```
$ python3 ./request-generator.py localhost:10000 20 10 10

Peticiones totales: 15770
Duración: 20.01 segundos
Tasa de peticiones: 788.15 peticiones/segundo
```

Ejercicio 4. Despliegue de cliente-servidor en nodos diferentes

En este ejercicio guiado usaremos dos nodos diferentes para ejecutar el cliente (*request-generator.py*), y el servidor (*multithreaded-server.py*). Para simular los dos nodos, crearemos dos contenedores Docker que estarán conectados usando una red privada. Los contenedores usarán la imagen de Docker `ray-os` ya creada, y compartirán con el *host* la carpeta *code*, que se encuentra en el directorio *ejercicio4*.

El primer paso consiste en copiar a la carpeta *code* los ficheros con el código Python de cliente y servidor, así como el fichero *temperatures.csv*.

A continuación, abriremos una shell en la carpeta del *ejercicio4*, crearemos una red privada de Docker llamada “test”:

```
$ cd <ruta_ejercicio_4>
$ docker network create test
206f74cd457685cb70c719b7330091f52579fce84bf2fbb6d5fe1bb18088f1d9
```

Crearemos el contenedor del servidor, llamado “python-server” con nombre de host “server”. Para ello, ejecutaremos el siguiente comando de docker que abrirá un shell dentro del contenedor:

```
$ docker container run -it --rm --name python-server \
    -h server --net test -v "${PWD}/code:/root/code" \
    ray-os /bin/bash
root@server:/#
```

Posteriormente volveremos a esa terminal para ejecutar el servidor.

Para lanzar el cliente abriremos otra terminal, accediendo al directorio del ejercicio 4, y ejecutaremos el siguiente comando de docker.

```
$ docker container run -it --rm --name python-client \
    -h client --net test -v "${PWD}/code:/root/code" \
    ray-os /bin/bash
root@client:/#
```

Con el comando anterior se crea un contenedor cliente llamado “python-client” y con nombre de host “client”.

Ahora que ambos contenedores están lanzados procedemos a arrancar el servidor:

```
root@server:/# cd /root/code
root@server:~/code# python3 ./multithreaded-server.py
Server listening on 0.0.0.0:10000
```

Y por último lanzaremos el cliente durante 20 segundos usando 10 hilos y 10ms de retardo entre cada petición. Para ello usaremos el siguiente comando, que asume que el servidor escucha en el puerto 10000 de la máquina “server”:

```
root@client:~/code# python3 ./request-generator.py server:10000 20 10 10

Peticiones totales: 15770
Duración: 20.01 segundos
Tasa de peticiones: 788.15 peticiones/segundo
```